

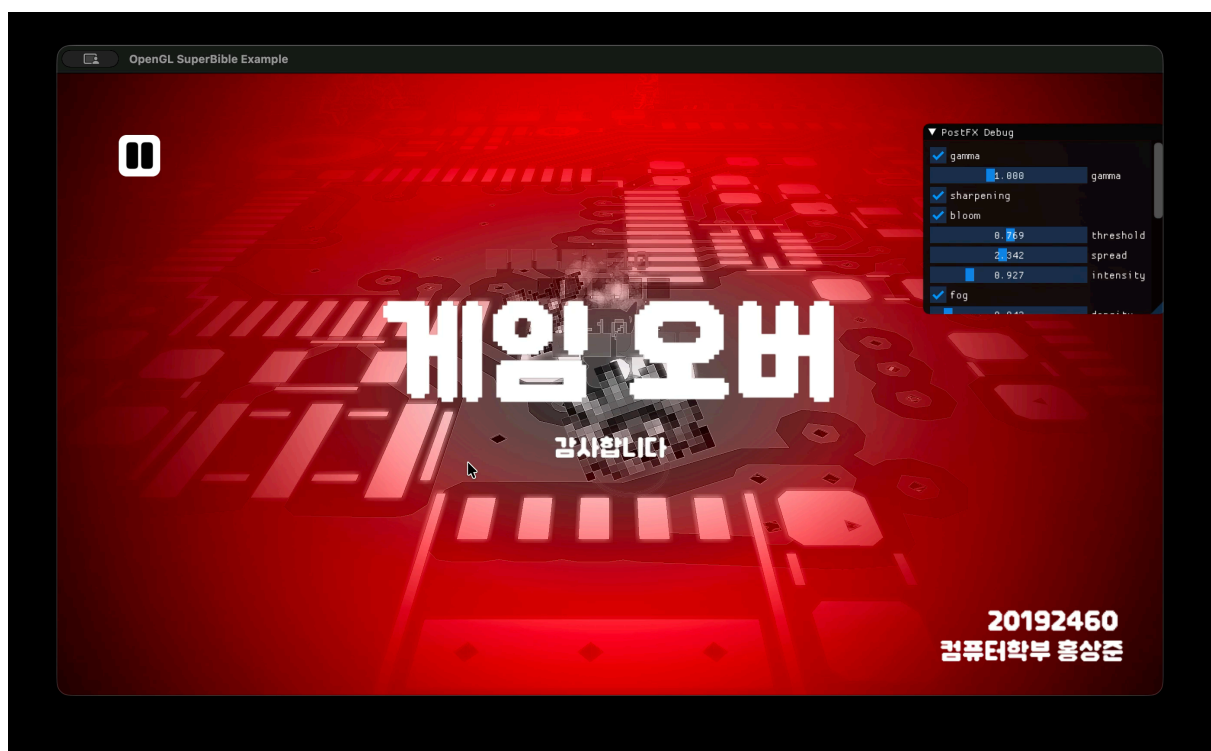
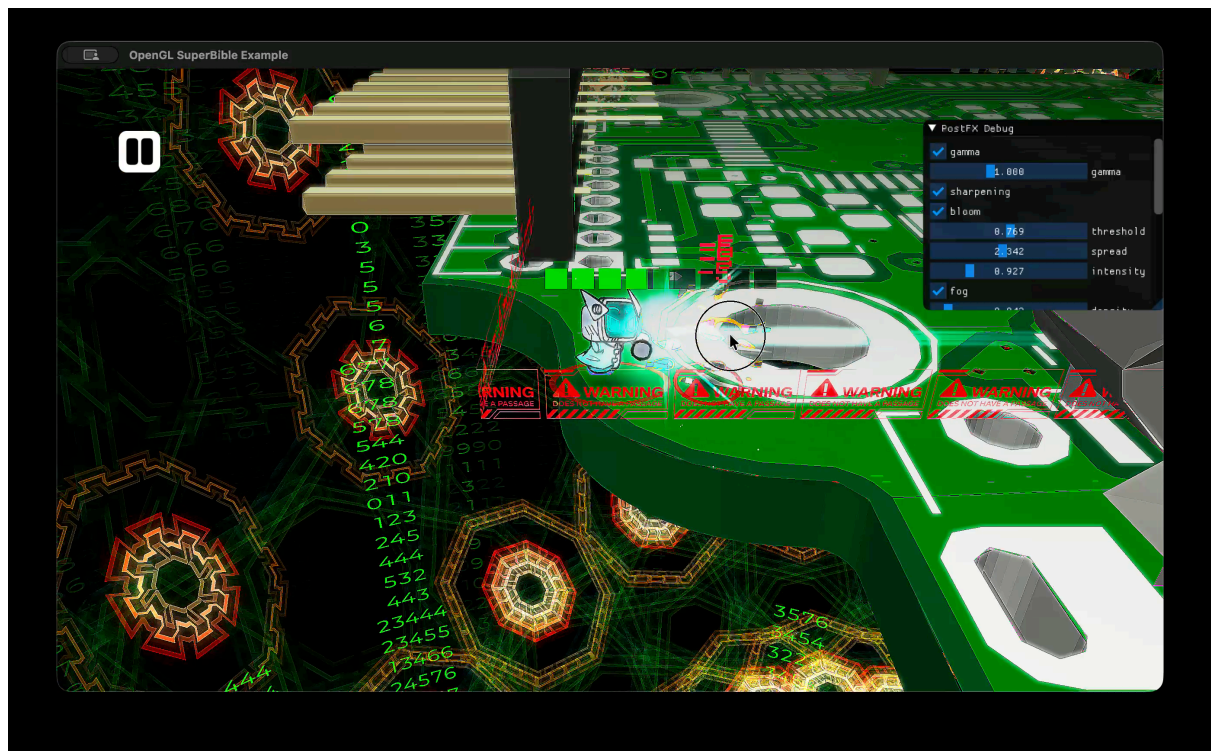
기말 프로젝트 보고서

이름	홍상준	학번	20192460
----	-----	----	----------

1. 프로젝트 개요

- 프로젝트명 : 버그 잡는 게임
- 프로젝트 레퍼런스 : 컬트 오브 더 램
 - 태그 : 2.5, 탐다운, 뱀서라이크, 태그 : 카세트 퓨처리즘, SF





2. 사용한 기법

강의에서 학습한 요소 중 사용한 것 모두 나열

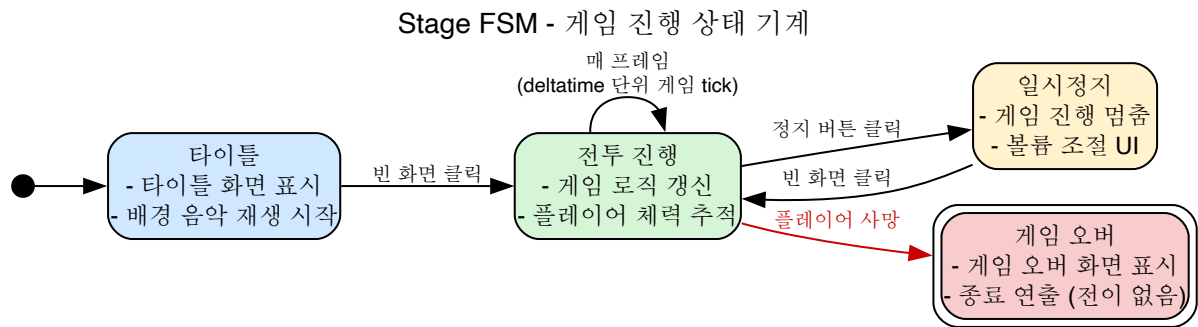
- Shader Program : 셰이더 컴파일,
CPU -> GPU 데이터 전송 (Vertex Layout, Uniform, Camera, Light, Texture)
- Resources : Obj/Fbx (Assimp), Texture Mapping,
- Postprocessing : Blurring(Bloom), Sharppening, Sobel, Fog(Depth Map)
- Buffers : VBO, EBO, RBO, FBO

강의 범위를 넘어선 시도 (있을 경우):

- Parametric Surface : 매끄러운 기하학적 Mesh 생성.
- Skybox : 구면좌표계 & 매트릭스 효과
- Postprocessing: Vignette, GrayScale
- Collision : AABB 충돌, 원 내접 외접. (Box2D)
- Particle System : Effekseer
- Tween Animation : Tweeny
- Sprite Atlas : UV Tiling
- Adaptive Audio System
- Shader VFX : Dissolve, Tint Highlight
- Font : Bitmap Font & Texture Renderer
- GUI : ImGui
- Scene Graph : Transform + Actor & Component

3. 핵심 구현 설명

3-1. [StateMachine + Stage]

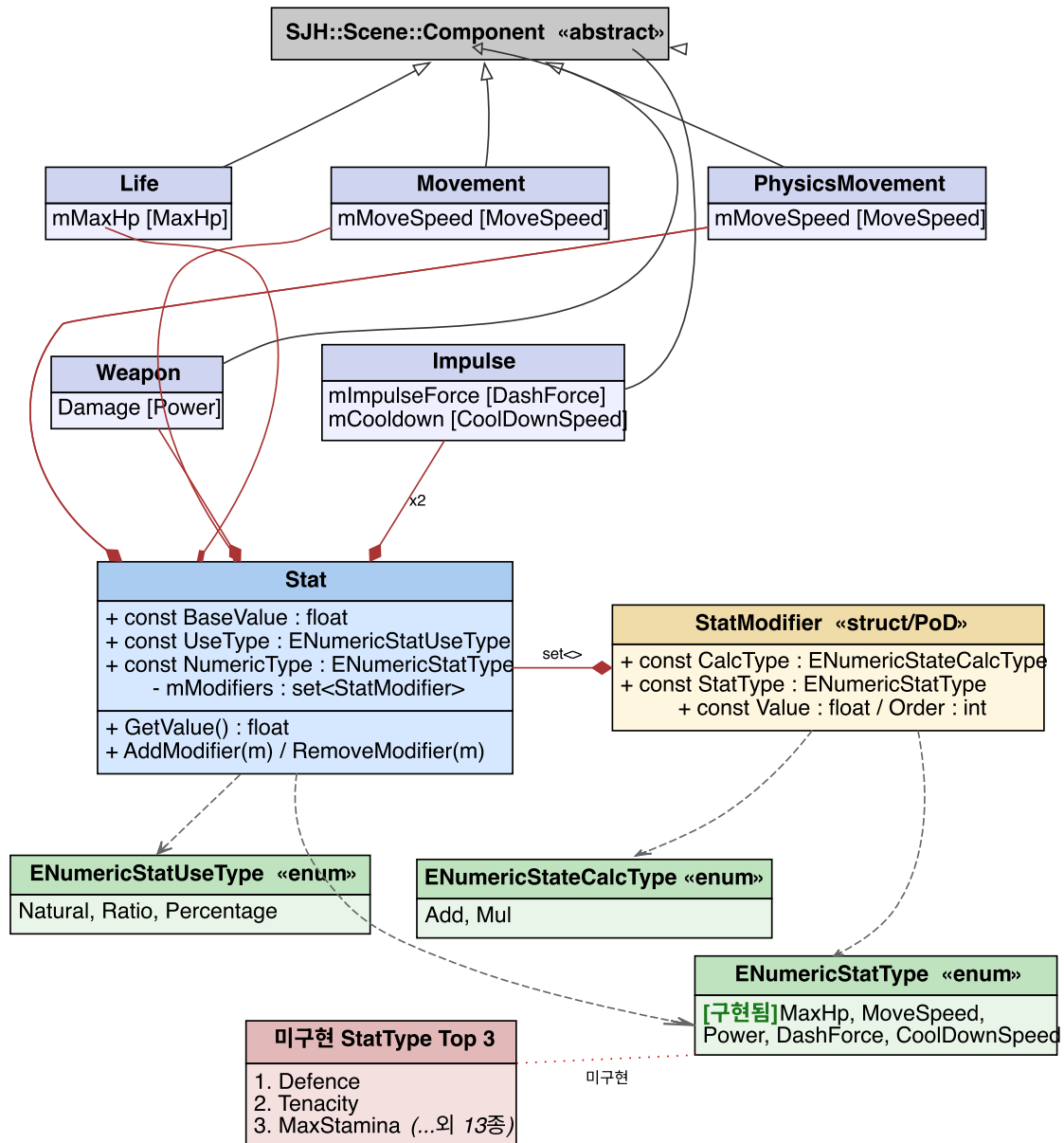


대표 모듈 : src/fsm, app/src/Stage/State

- Sb7 render 함수의 단일 사이클에 모든 기능을 모아 두던 방식을 버리고, bool 판정 대신 상태라는 단위로 동작을 분할하여 유사 기능끼리 응집시켰다.
- 상태와 상태 전이는 정수형 플래그의 조합으로 구성해, 상태 전이 1 대 N 매핑과 전이 불능 가드를 모두 수월하게 처리한다.
- StateMachine 모듈은 추후 복잡한 Entity AI(Boss) 제작에도 재활용할 수 있다.

3-2 [Algebraic +Stat]

Algebraic Stat 시스템 + 구체 컴포넌트 - 클래스 다이어그램

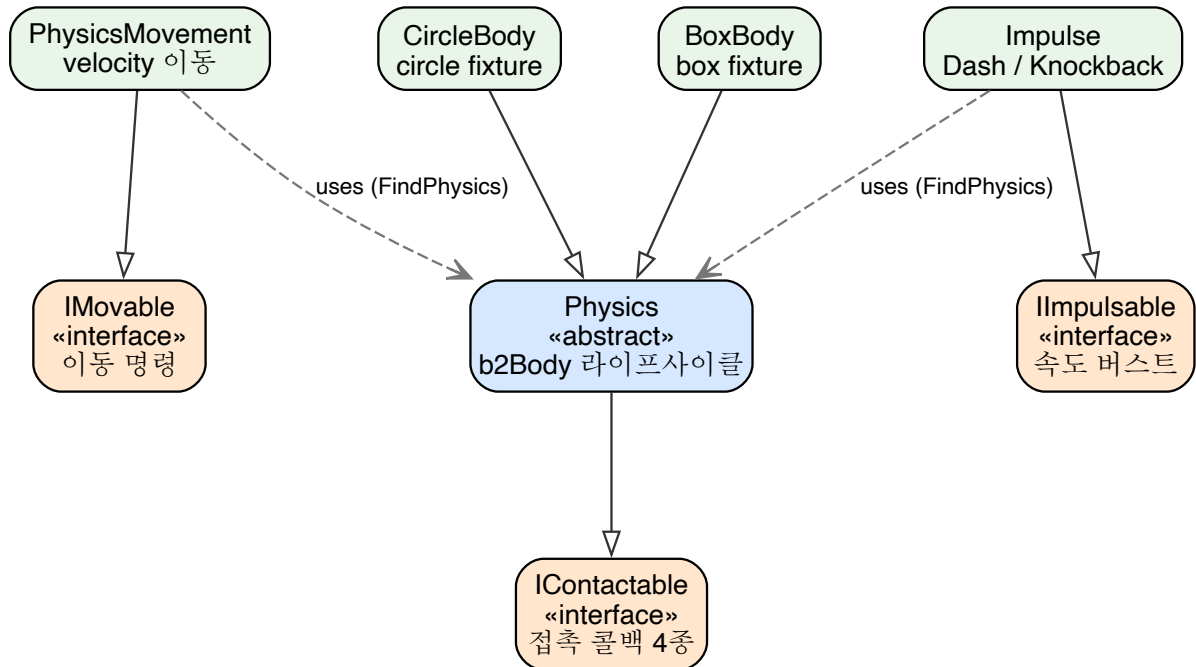


대표 모듈 : app/src/Algebraic

- 런타임에서 버프 적용 후 적용 해제 순서가 어긋나 값이 영영 틀어지는 문제를 해결하려고, Stat 과 연산자, 피연산자 그룹의 Modifier 로 객체를 분리했다.
- 값을 안전하게 올바른 순서로 계산하기 위해 값이 변경되면 재계산하되, 재계산이 끝나면 DirtyFlag 로 값을 캐싱해 접근을 빠르게 한다.

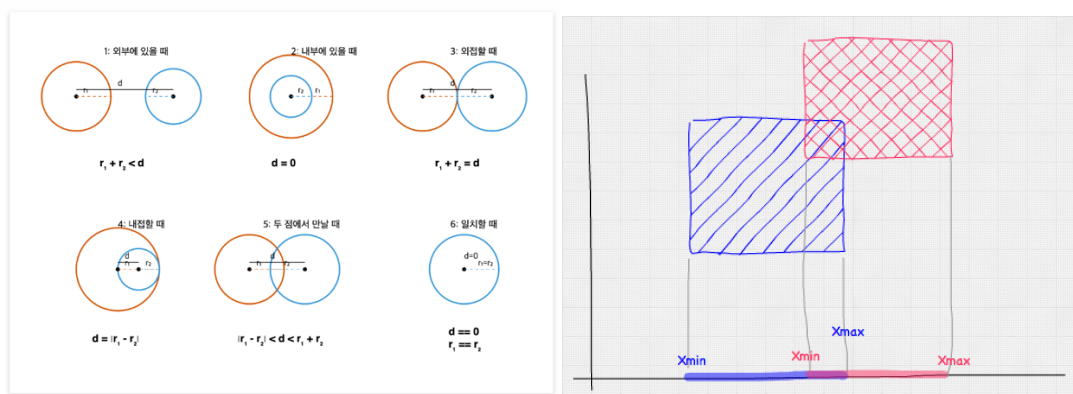
3-3. [Physics + Body.Impls]

TopdownShooter::Physics - 추상/구체 클래스 (핵심만)

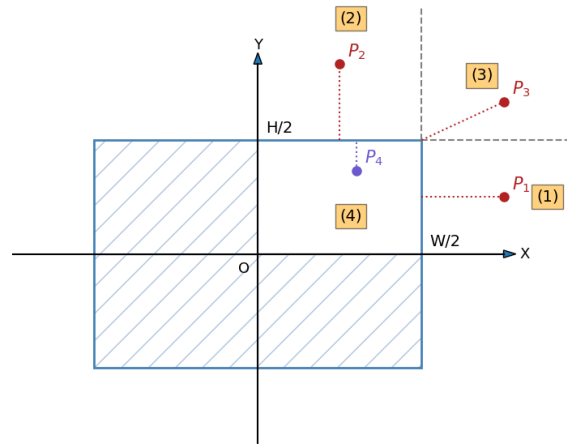


대표 모듈 : Box2D, app/src/Physics

- Unity 의 Component 로직을 벤치마킹해, b2Body 의 Velocity 로 물리 기반 객체 움직임을 구현하고 Impulse 를 적용한다(양 = dash, 음 = knockback).
- **Box2D 모듈 분석** - 원과 원의 위치 관계 및 AABB 박스 충돌.



- **Box2D 모듈 분석** - 원과 박스의 충돌은 원의 중심과 (2), (3), (1) 각 축 중 거리가 최소인 점을 비교해, 그 거리가 반지름보다 작으면 충돌로 계산한다.



3-3. [SpriteAtlas + UV Splitting Shader]

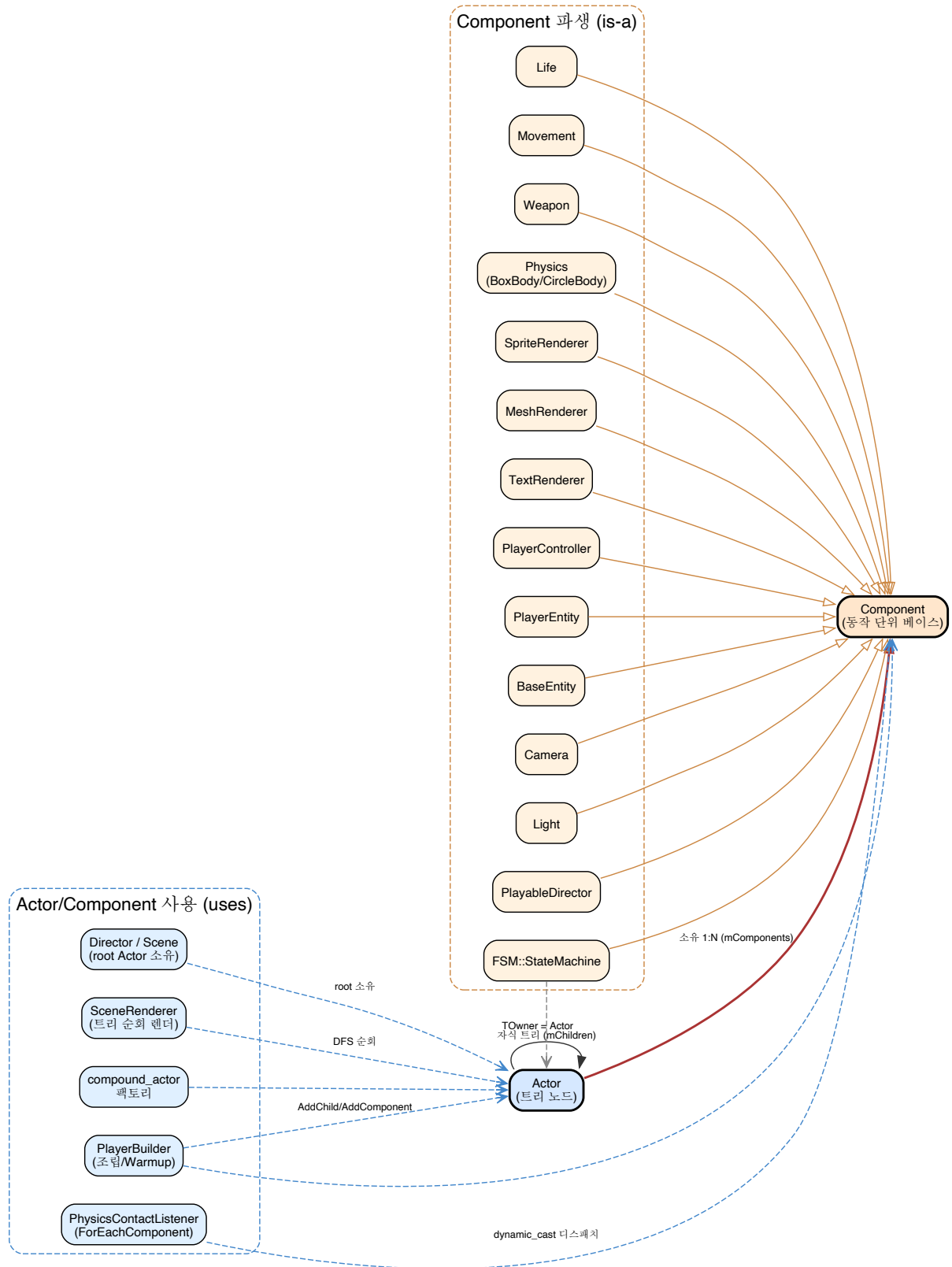


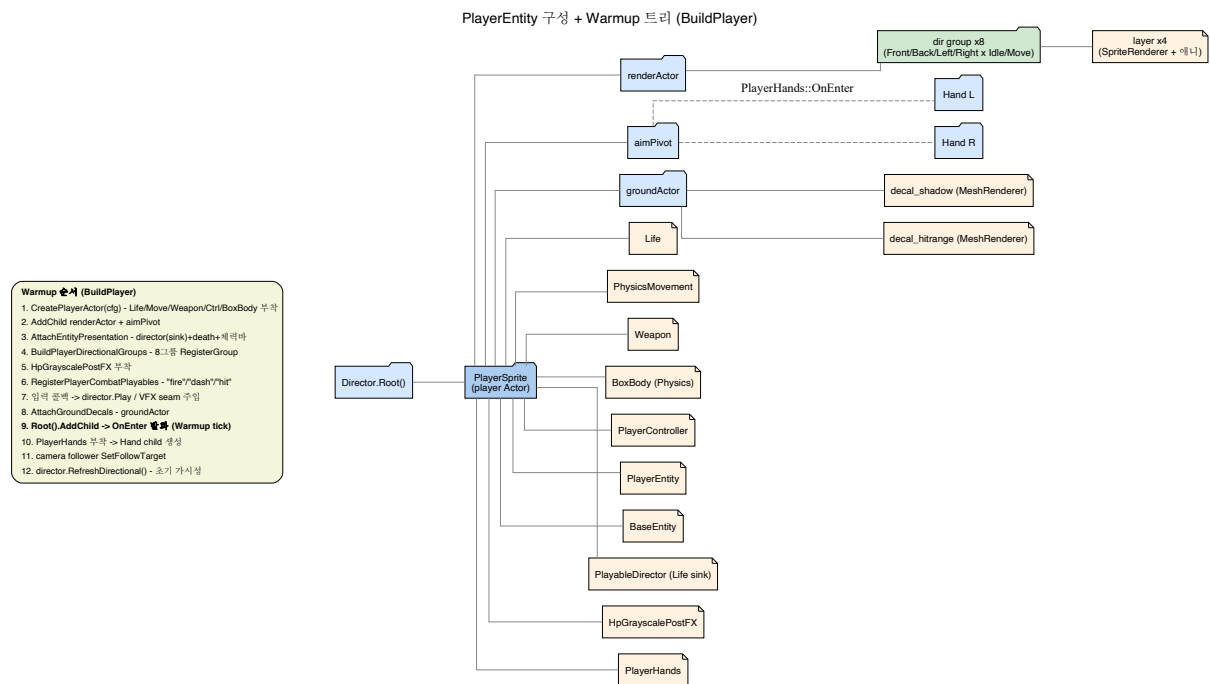
1	2	3	4
5	6	7	8
9	A	B	C
D	E	F	0

- 공통점부터 정리하면, sprite 와 bitmap font, skybox 는 모두 텍스처를 균일한 격자로 나눈다는 점에서 같다. $[0, 1]$ 구간을 $cols \times rows$ 칸으로 나눠 하나의 텍스처에 배치하도록 구성했다.
- Sprite 격자 설정은 Builder 패턴으로 `UniformAtlas.SetGrid(cols,rows)`를 체이닝하도록 API 를 구성했다. `BitmapFont` 는 xml 형식에 맞춰 $cols = scaleW / cellW$ 로 같은 격자를 인식하도록 구현했고, 두 공통점은 인터페이스로 묶을 예정이다.
- 정점 셰이더는 시작점 + 지역좌표 $\times$ 칸 크기로 나눠 UV 의 $[0, 1]$ 구간을 CPU 에서 Uniform 으로 받아온다(matrix rain skybox 셰이더는 교안을 따라가다 보니 하드코딩으로 작성했다). 프래그먼트가 vUv 로 텍스처를 읽어 칸의 픽셀만 화면에 칠한다(`texture(uAtlas, vUv)`).
- 결국 숫자 4 개($uUvRect$)만 바꾸면 같은 텍스처에서 다른 칸(애니 프레임이나 글자)으로 교체된다.
- Sprite Flip 은 UV 범위를 뒤집느냐 마느냐로 구현했고, 이때 Front & Back Cull 문제로 뒤집힐 때 투명해져 양방향 컬링으로 설정해 해결했다.

3-4. [SceneGraph + Actor & Component]

SJH::Scene::Actor / Component 에 의존하는 객체들





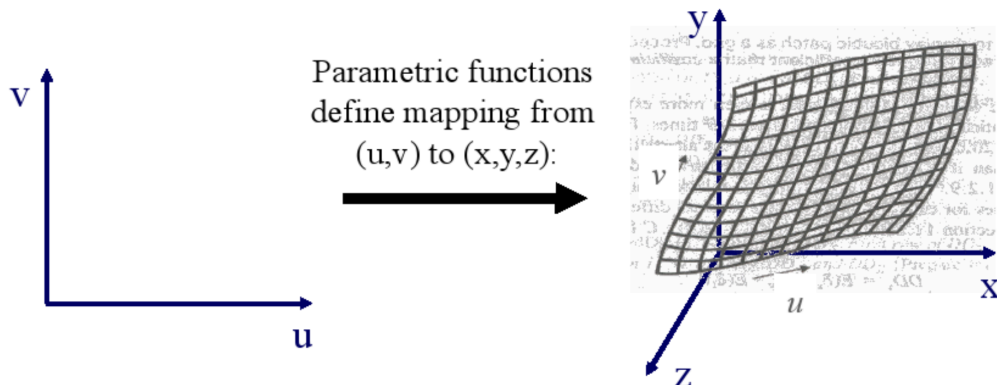
대표 모듈 : src/scene (actor), src/object (transform)

1. Transform(src/object)은 로컬 TRS(Translate * EulerRot * Scale) 값만 갖고 GetLocalMatrix()로 로컬 행렬을 만든다. 계층, 월드 합성은 Actor 가 맡아, GetWorldMatrix()가 부모(mParent) 월드 행렬에 자기 로컬을 곱하는 재귀로 루트까지 행렬을 체이닝한다 - 부모가 움직이면 자식이 따라온다.
2. Actor 는 상속을 금지(class FooActor : public Actor 형태 금지)하고, 특수 속성은 Component(OnEnter/OnExit/Update 생명주기)를 부착해 부여한다. AddComponent<T> / GetComponent<T>(Unity GetComponent 정통)로 다뤄 클래스 폭발 없이 유연하다.
3. 자주 쓰는 Actor+Component 조합은 compound_actor.h 의 free factory(CreateCameraActor 등)로 묶어 호출부를 간결히 했다. 향후 ImGui 엔진 에디터와 JSON 직렬화로 씬 구성을 코드와 분리(Data-Driven)할 계획이다.

Parametric Surfaces

- Defined by parametric functions:

- $x = f_x(u, v)$
- $y = f_y(u, v)$
- $z = f_z(u, v)$



FvDFH Figure 11.42

1. 구, 원기둥, 원판(Disk) 정점 구성을 자동화 하기 위해 도입
2. Surface Function 동일한 u, v 가 들어오더라도, 모델의 모양은 천차만별

$$x = f_x(u, v) \quad y = f_y(u, v) \quad z = f_z(u, v)$$
3. 정의역(u_{start}, u_{end} , v_{start}, v_{end})과 해상도(u_{res}, v_{res})를 인자로 받는다.

- i. 각각의 정의역을 추가로 입력해야 한다.
 - $u \in [u_{start}, u_{end}]$
 - $v \in [v_{start}, v_{end}]$
- ii. 얼마나 촘촘히 쪼갤지(iterate)를 정하는 해상도 값을 입력한다.
 - $u_{res} \in [0, \infty)$
 - $v_{res} \in [0, \infty)$
 - 이게 곧 Quad 메시의 개수가 된다.

$$u_{res} \times v_{res} = \text{Quads Count}$$

4. Sphere 의 SurfaceFunction ($u \rightarrow$ 경도, $v \rightarrow$ 위도)

- SurfaceFunction 은 (u, v)를 받아 표면의 한 점을 돌려주는 함수다. Sphere 를 예로 들면 v 는 위도 $lat = v \cdot \pi - \frac{\pi}{2}$, u 는 경도 lon 으로 매핑되어 위치는 $(R \cdot \cos(lat) \cdot \cos(lon), R \cdot \sin(lat), -R \cdot \cos(lat) \cdot \sin(lon))$ 이 된다.

5. 모델링

정점 벡터 생성하기 : $u_{res} \times v_{res} = \text{Quads Count}$ 이므로
정점들은 그보다 하나씩 더 많아야 한다.

1. $\Delta u, \Delta v$ 생성

$$\Delta u = \frac{u_{end} - u_{start}}{u_{res}}$$

2. 이터레이팅

i. position : $u_{start} + \Delta u * \text{IterCount}$

- u, v 별로 $\Delta u, \Delta v$ 만큼 한 칸 한 칸씩 위상을 옮겨가면서
정점 데이터를 쌓아나가면 된다.

ii. texCoord : 텍스처 uv 좌표는 정규화된 값을 가지고 있으므로
정의역은 $[0, 1]$ 이다.

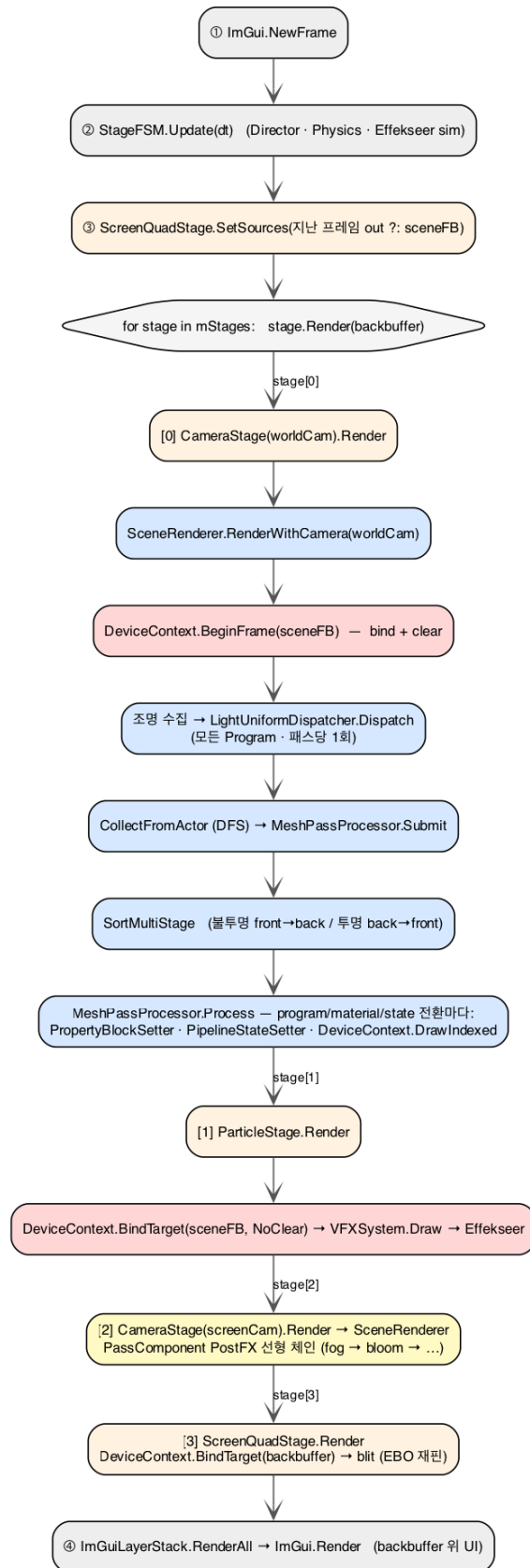
- 따라서 $\frac{\text{IterCount}}{\text{Resolution}}$ 이 UV 텍스처 좌표가 됩니다.

iii. normals - 두 접선 벡터(= 편미분)를 외적해 법선을 구하는 것이
일반적이지만, Sphere 는 중심이 원점이라 정점의 위치 벡터를
정규화하면 그대로 법선이 되어, 외적 없이 해석적으로 구한다.

$$S_u \approx \frac{S(u + \epsilon_u, v) - S(u, v)}{\epsilon_u}, S_v \approx \frac{S(u, v + \epsilon_v) - S(u, v)}{\epsilon_v}, \vec{n} = \frac{S_u \times S_v}{\|S_u \times S_v\|}$$

3-6. [Renderer & Multi Pass Rendering]

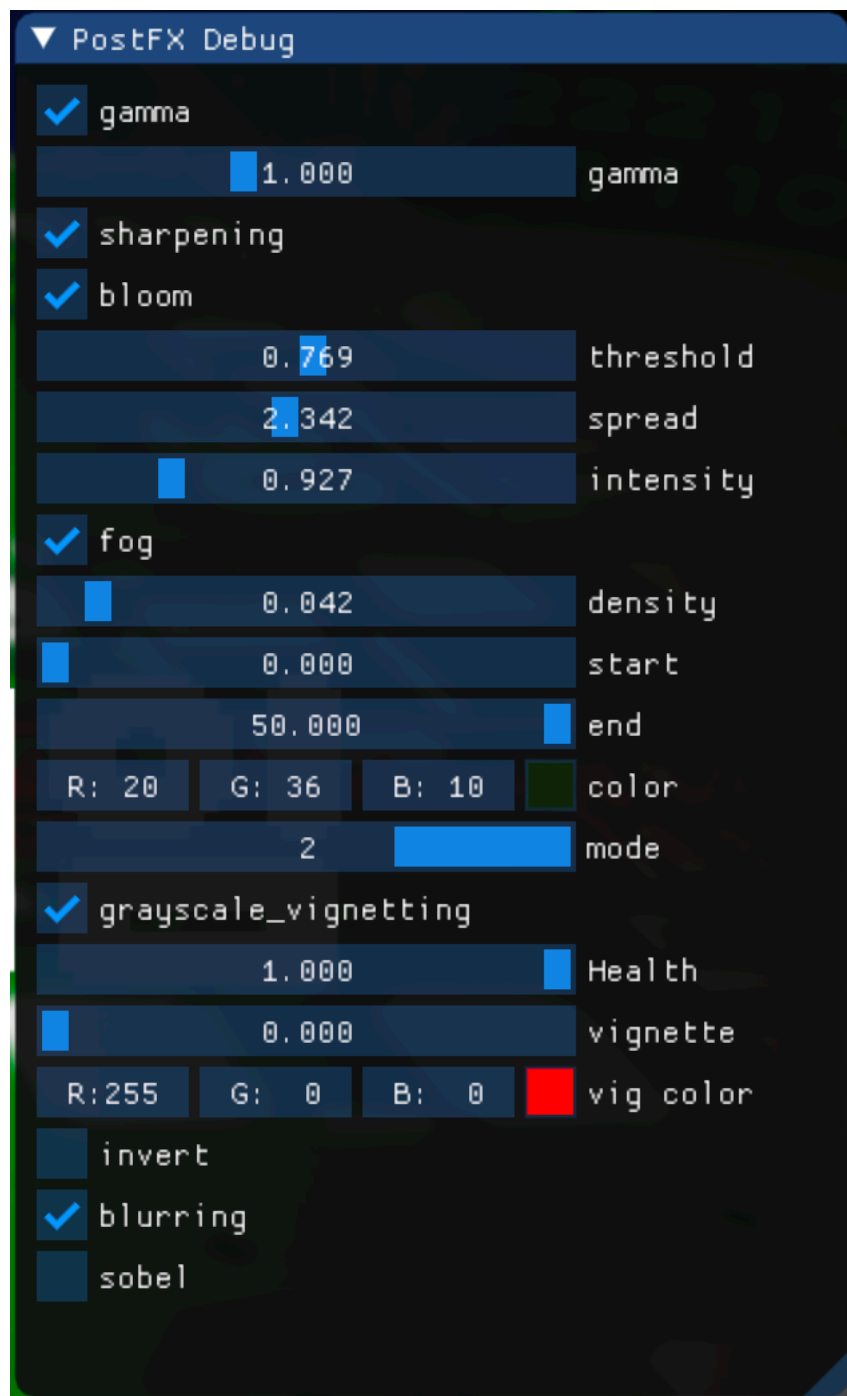
3-6 한 프레임 render() 호출 시퀀스



대표 모듈 : src/render (scene_renderer, mesh_pass_processor)

1. 전제 - 카메라마다 앞에 렌더 타겟(FBO=텍스처)이 한 장 붙어 있고, 그 카메라가 그리는 모든 것은 화면이 아니라 그 텍스처에 먼저 그려진다
Unity Camera.targetTexture 를 벤치마킹 하였음
2. 멀티패스는 Application 이 든 vector<stage> 컨테이너를 순서대로 실행하는것.
worldCam → Effekseer → screenCam(후처리) → 최종 화면 출력.
3. 렌더링에 앞서 SceneRenderer 가 sceneFB 에 그릴 대상 정보를 수집한다 - 조명은 Director 컨텍스트에서 켜진 DirLight & PointLight & SpotLight 만 골라 모으고, 그릴 물체는 Actor 트리를 DFS 로 순회하며 MeshRenderer 가 든 Mesh&Material 을 DrawCommand 로 큐에 담고, 수집한 DrawCommand 는 Pass 종류(불투명,투명)와 카메라 거리로 정렬해 GL 상태 전환이 최소가 되는 순서로 그린다.
4. 그리기 직전, 잘 안 바뀌는 데이터(조명)는 패스당 1 회, 자주 바뀌는 것(모델 행렬)은 *실제 현 프로젝트에서는 라이트가 움직이는 기교가 필요없었음..*
메시마다 셰이더로 보내 GL 상태 전환을 최소화하였다.
5. Postprocessing 는 카메라 앞 사각형에 직전 결과 텍스처를 붙여 다시 찍는 것으로, PassComponent 의 InputFB & OutputFB 가 같은 포인터를 공유하도록 구성함.
패스당 FBO 1 개씩 인풋이 곧 이전 패스의 결과를 연달아 받도록.
6. Effekseer, ImGui 같은 외부 모듈도 같은 렌더링 프로세스를 따르도록 AI 도움을 받음. Effekseer 는 ParticleStage 로 따로 버퍼를 나누어 sceneFB 에 합성하고, ImGui 는 모든 패스가 끝난 뒤 백버퍼에 직접 엮는다.

3-7. [Shader VFX & Postprocess & Skybox + 매트릭스 효과]



대표 모듈 : apps/_MyApp_/resources/shaders

(postprocess, healthbar, matrix_skybox)

1. Fog: 깊이 맵(uDepth)의 비선형 NDC 깊이를 역투영(uInverseProjection)해 카메라-픽셀 실제 거리를 복원하고, 그 거리로 안개색을 블렌딩 함(far plane 하늘은 제외).

2. Grayscale: 휘도 $\text{dot}(\text{rgb}, (0.299, 0.587, 0.114))$ 로 무채색을 만들어 원본과 블렌딩
3. Vignette: 화면 중심거리 d 로 가우시안 마스크 $\exp(-k \cdot d^2)$ 를 만들어 중앙=장면, 외곽=비네트색으로 블렌디 (둥근 원형 스텐실).
4. Bloom: 휘도로 밝은 픽셀만 step으로 골라 9×9 박스 블러로 번지게 한 뒤 원본에 더한다(additive glow).
5. HealthBar: 가로 좌표 u 를 $\text{fract}(u \cdot N)$ 으로 N 조각으로 쪼개고, smoothstep과 fwidth로 조각 틈 경계를 줌과 무관하게 1픽셀 폭으로 매끈하게(안티에일리어싱) 그린다.
6. Skybox 영화 매트릭스 효과: 방향벡터를 구면좌표($\theta = \text{atan2}(z, x)$, $\varphi = \text{asin } y$)로 역매핑해 $UV(\theta/2\pi, \varphi/\pi)$ 를 얻고, floor/fract로 글자 격자를 잘라 칸을 채운다.

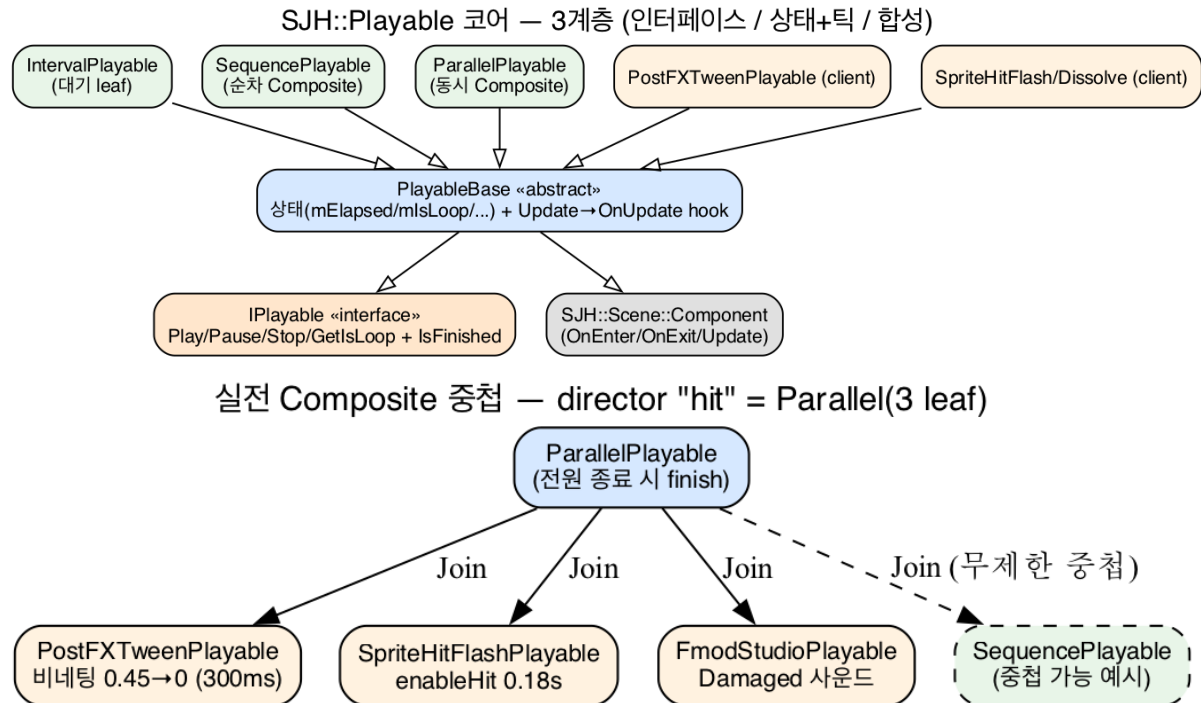
3-8. [Adaptive Audio System & FMOD]



1. FMOD 측의 Parameters 라는 요소를 통해, 시퀀서들에 포함된 오디오들을 오토메이션(Low Pass Filter 적용 미적용), 자연스러운 음악간 블렌딩을 적용하도록 세팅함. 즉 코드와 독립적으로 주로 아티스트와 개발자간 협업을 돕는 툴을 적용함.

3-9. [Playable]

대표 모듈 : src/playable, apps/_MyApp_/src/Playable



1. 피격 시 실행되는 연출은 (빨간색 비네팅 & 플레이어 플래시 & 피격 사운드 등) 한꺼번에 터지고, 사망하면 디졸브에 이어 폭발 FX 가 차례로 일어난다. 이렇게 동시에 또는 순서대로 겹쳐 일어나는 연출을 다루려고 Playable 이라는 기능을 도입했다. 이 내용은 Unity DOTween 빌더 패턴을 벤치마킹함.
2. 컴포넌트마다 if(피격){타이머++}로 풀어 놓으면 연출 하나를 추가할 때마다 모든 곳에 분기가 늘어 조합이 감당하기 어려워지고, 게임 로직(HP 계산)과 연출(깜빡임)이 한 함수에 엉키게 되어 결합도가 증가하는 문제가 있었음.
3. 제작 철학은 총 3 계층으로 IPlayable 이라는 인터페이스로 접근하여, 컨테이너 관리가 되도록 하였음. Play/Pause/Stop/Loop 와 IsFinished 만 노출한다.
4. 합성(Sequence=순차, Parallel=동시)은 모두 공통적으로 IPlayable 포인터를 사용하기 때문에 Sequence 안에 Parallel 을 일괄적으로 처리해도 문제없게 구현하였다.
5. 연출을 추가할 때는 Append, Join 등의 빌더 함수를 이어 붙여(메서드 체이닝) 각 연출이 타이밍에 맞게 수행되도록 직접 기입한다.
Append 는 앞 연출이 끝난 뒤에 잇고, Join 은 직전 연출과 동시에 묶는다.

4. AI 사용 내역

사용 부분	사용한 AI	활용 방식	본인이 수정/이해한 내용
Camera 행렬 (뷰&역투영&affine 역)	Claude Code	역행렬 닫힌 해 유도 구현 보조	역투영, affine inverse 식과 column-major 규약 직접 검증, 코드 작성, 디버깅
플레이어 손	Claude Code	scene graph WorldMatrix 합성 로직 골격	손 벌림각 파라미터 직접 튜닝
테스트 리팩토링 설계, API 문서	Claude Code	brainstorming·spec 레퍼런스 정리	설계 의사결정 직접, 문서 초안 검토 및 수정
레퍼런스 코드 분석	Claude Code	깃허브, 블로그의 VFX 셰이더 코드 해설.	셰이더 사이드 Font UV 타일링 기법,
외부 모듈 API 분석	Claude Code	Context7 을 통해 API 의 사용 방법과 예제 제공 요구 Box2D, Effekseer, FMOD Core 등..	본인 프로젝트 런타임 언어 C++ 에 적합한 함수 호출, 시그니처 등을 이해함.
CMake 외부 모듈 설치 크로스플랫폼 세팅	Claude Code	MSVC, Ninja 빌드를 구성하는 빌드 제네레이터 CMake 세팅법 도움을 얻음 또한 GithubAction 을 통한 Window Build CI/CD 구성	크로스 플랫폼 세팅을 위한 C++ DEFINE 설정과, 외부 모듈을 자유롭게 사용하기 위한 CMake 함수와, 톨체인 이해

5. 레퍼런스 내역

- HPBar : <https://github.com/Sam-Schiffer/UnityCircularHealthBarsLite>
- Fog Postprocessing : <https://github.com/hughsk/glsl-fog>
- 매트릭스 효과 스카이박스 : <https://godotshaders.com/shader/matrix-rain/>
- Actor - Component
 - https://docs.cocos2d-x.org/api-ref/cplusplus/v3x/d3/d82/classcocos2d_1_1_node.html
 - <https://docs.cocos.com/creator/2.4/api/en/classes/Component.html>
 - <https://github.com/Unity-Technologies/UnityCsReference/blob/master/Runtime/Export/Scripting/Component.bindings.cs>

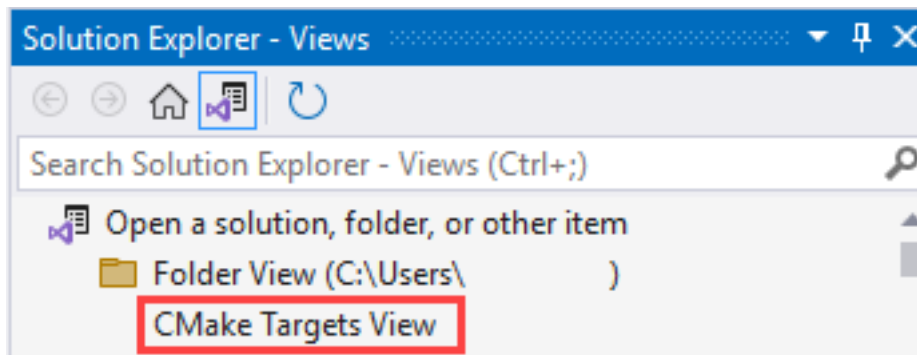
6. 프로그램 실행 방법

1. 바로 실행하기

- `./ build_msvc/apps/_MyApp_/Release/_MyApp_.exe`

2. Visual Studio 에서 CMake 보기 -> msvc-20XX 선택

-> 캐시 삭제후 빌드 (약 20 분 내외로 걸릴것으로 예상) -> _MyApp_.exe 실행



- <https://learn.microsoft.com/ko-kr/cpp/build/cmake-projects-in-visual-studio?view=msvc-170>